Ordenamiento - Sorting

Algoritmos y Estructuras de Datos

Departamento de Computación,
Facultad de Ciencias Exactas y Naturales,
Universidad de Buenos Aires

4 de noviembre de 2026

Autor/adaptación: Camilo Semería

Motivación

¿Por qué sorting?

- Es uno de los **problemas clásicos** de computación, del área de algoritmos.
- Buscar elementos en una estructura es mucho más eficiente si está ordenada.
- Es muy común que un algoritmo ordene algo como subrutina, y así su eficiencia está atada al algoritmo de sorting que use.
- Tener buenos algoritmos de sorting puede ayudar a resolver muchos problemas en forma eficiente.

Un pequeño repaso

- **Insertion Sort, $\Theta(n^2)$** en el peor caso.

  Cada elemento de intercambia hacia izquierda hasta encontrar uno ≤. Por cada elemento intercambioa máximo n veces. **Es estable**.

- **Selection Sort, $\Theta(n^2)$**

  Busca el mínimo elemento posterior a la posición i y lo **intercambia con A[i].** Buscar el menor cuesta $O(n)$ y son n elementos. **No estable**.

- **MergeSort, $\Theta(n \log n)$**

  Algoritmo “recursivo” de la familia Divide&Conquer (más info en TDA/Algo3) que ordena sus dos mitades y luego las fusiona. **Es estable.**

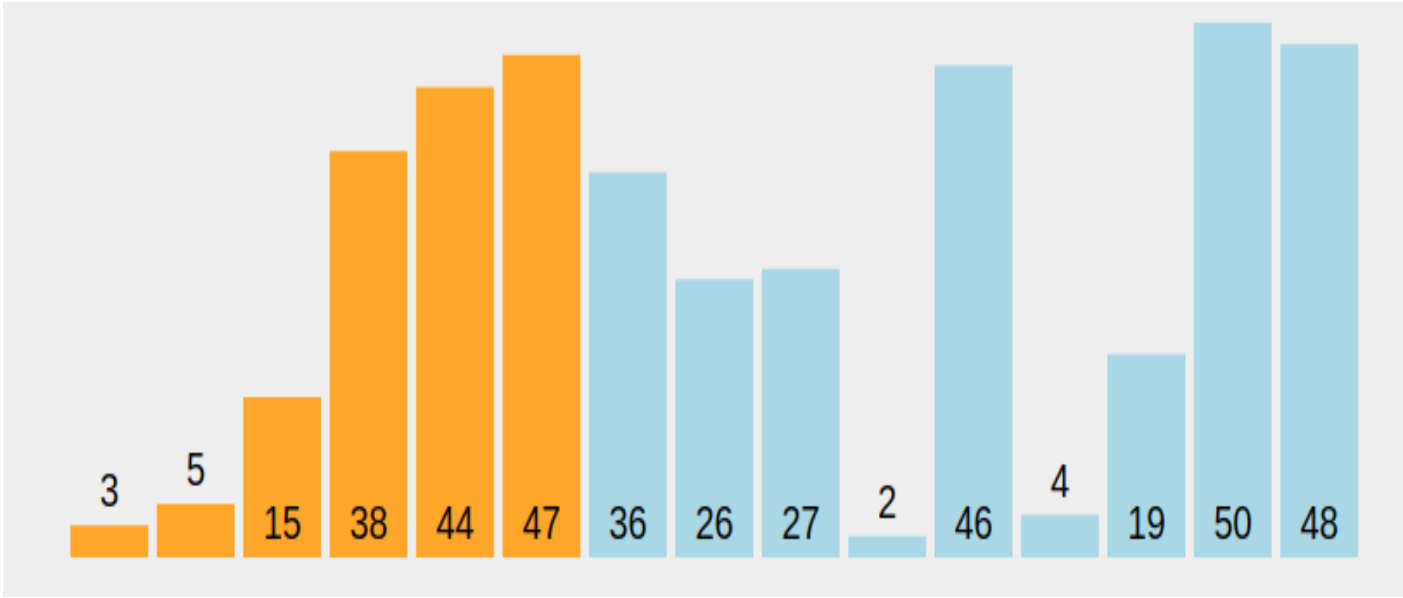
- **QuickSort, $\Theta(n^2)$** en el peor caso. **$\Theta(n \log n)$ en el caso promedio**.

  Elige un pivote y separa los elementos entre los menores y los mayores a él. Luego, se ejecuta el mismo procedimiento sobre cada una de las partes. Muy bueno en la práctica. **No estable**.

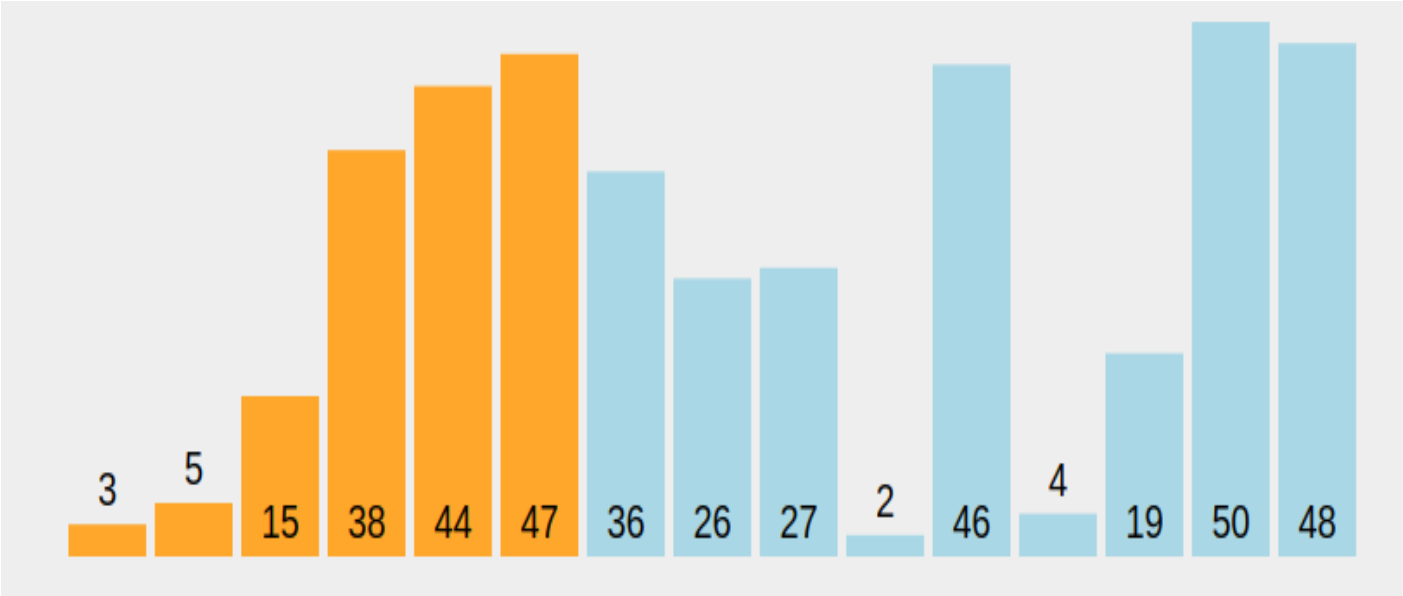
- **HeapSort**, $O(n + n \log n) = \boldsymbol{O(n \log n)}$

  Arma el Heap en $O(n)$ y va sacando los elementos ordenados en $O(\log n)$. Funciona Como una cola de prioridad. **No estable.**

Trivia - Adivinen qué algoritmo se está usando

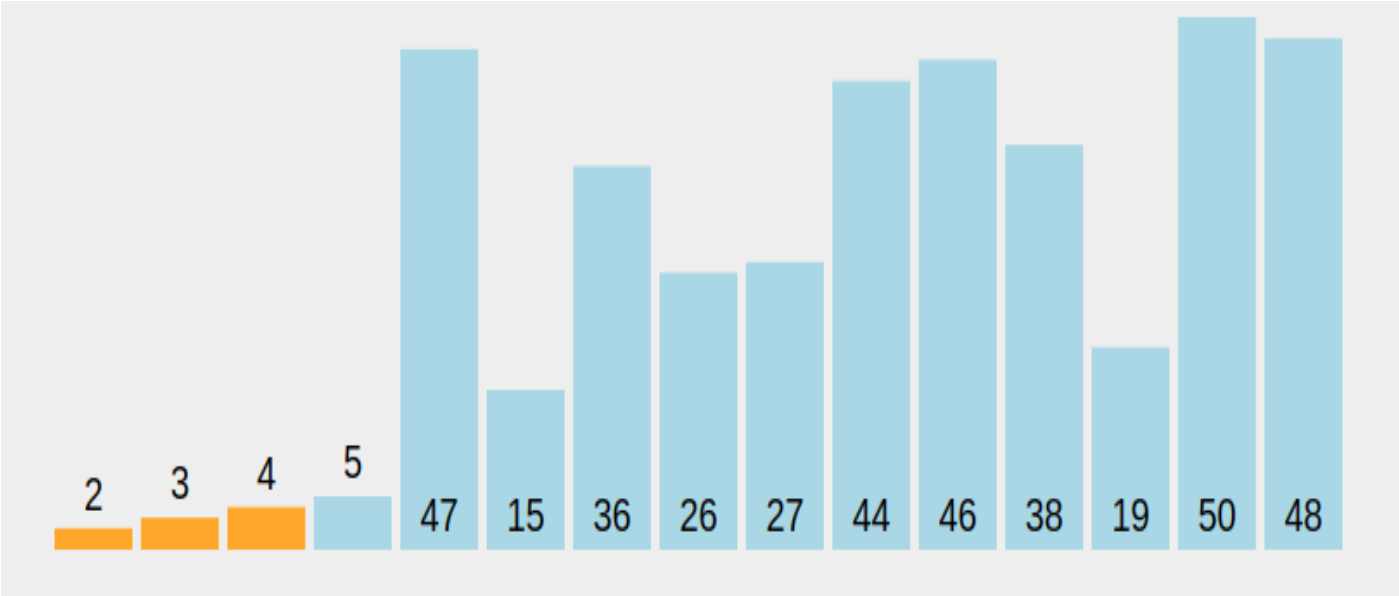


Trivia - Adivinen qué algoritmo se está usando

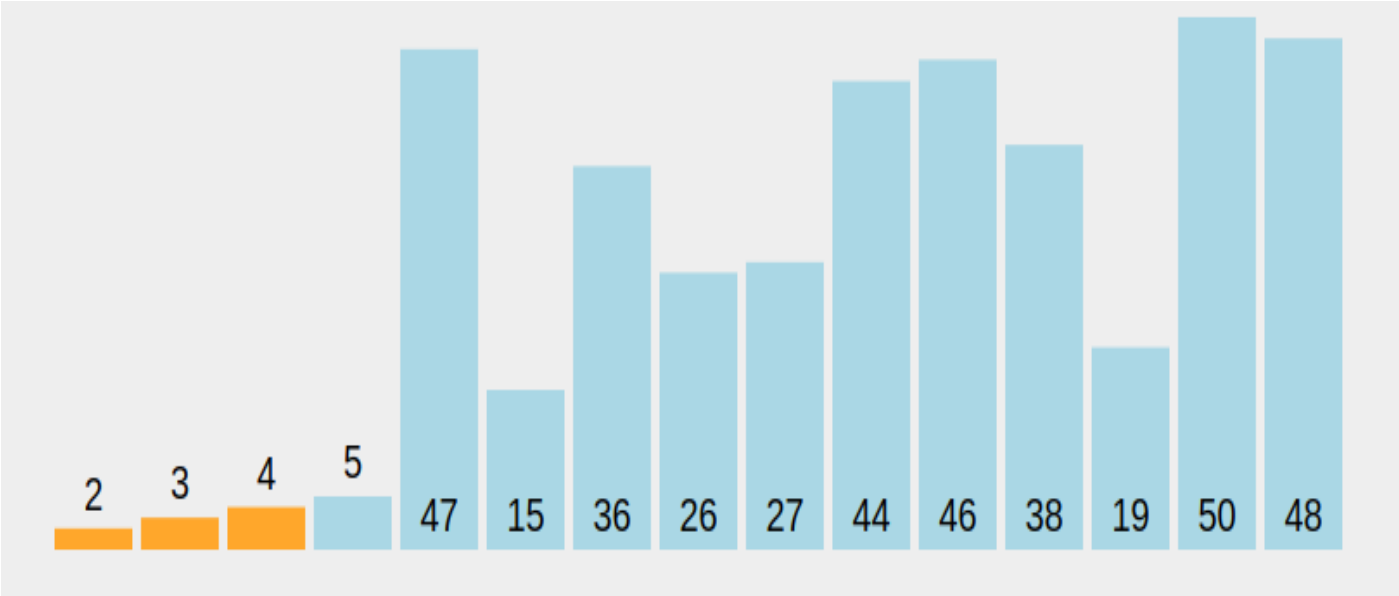


Insertion Sort

Trivia - Adivinen qué algoritmo se está usando

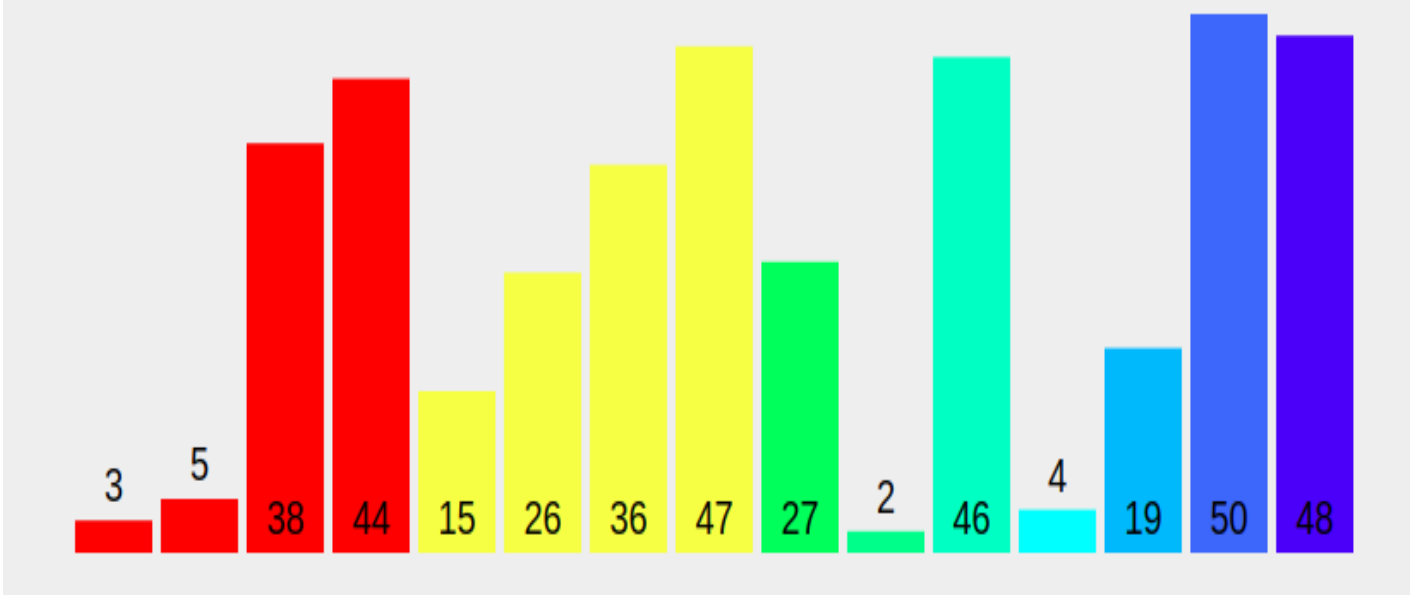


Trivia - Adivinen qué algoritmo se está usando

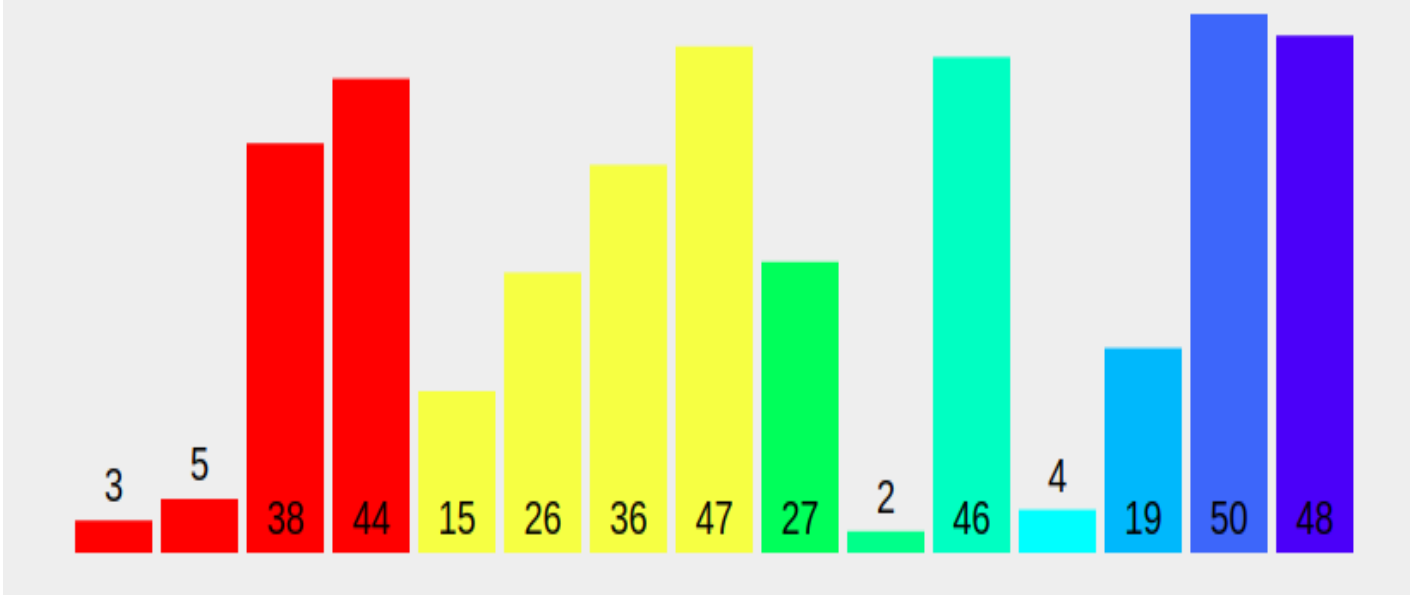


Selection Sort

Trivia - Adivinen qué algoritmo se está usando



Trivia - Adivinen qué algoritmo se está usando



Merge Sort

Sintaxis

```
Proc xSort(A: array<T>, fVaPrimero: T x T -> T)
```

- Los algoritmos de ordenamiento por comparación reciben un array (o un vector) y una función de comparación.
- El arreglo/vector pasado como parámetro se modifica (es inout).
- La función de comparación recibe dos elementos del mismo tipo del arreglo y devuelve el que va primero según nuestro criterio de orden.
- **No hace falta implementar dicha función**, con usar un nombre declarativo o escribir un comentario del criterio es suficiente.

Ejemplos

```
//B es array de enteros
mergeSort(B, deMenorAMayor)

//C es vector de tuplas<int, string>
heapSort(C, fVaPrimero)
// Ordena de forma decreciente según
// primer elemento de la tupla
```

Reflexionando sobre lo hablado

1.¿Qué algoritmo de los vistos creen que podría usar un niño de 6 años para ordenar objetos? ¿por qué?

2.¿Alguno de los algoritmos tiene un mejor caso? ¿cuál?

2b.¿Eso le suma utilidad al algoritmo?¿Por qué?

3. Mergesort tiene un paso llamado “merge” ¿de que se trata? ¿es en si mismo un algoritmo útil?

4.¿Por qué creen que en la materia amamos MergeSort?

5,¿Cómo funciona el algoritmo de Floyd/Heapify?

6.Tengo un criterio principal y uno de desempate. ¿Qué algoritmo usarían para ordenar en dos pasadas?

7. ¿Que criterio usarían primero en el caso anterior?

8.¿Que algoritmo usarían si la entrada suele venir en orden inverso? ¿Y ordenada?

Algoritmos no comparativos

- Bucket Sort
- Counting Sort
- Radix Sort

Reflexión

- Estos algoritmos ordenan arreglos comparando los elementos, es decir, son algoritmos de sorting por comparación.
- En el peor caso son $\Omega(n \log n)$ por **Lower Bound**.
- ¿Puede haber algo mejor?
- Sí, por ejemplo, si tenemos información de los elementos.
- Tambien podríamos tener complejidades sujetas a otros factores, más allá de la cantidad de elementos de la entrada.

Bucket Sort

- **Bucket Sort** ordena arreglos de cualquier tipo.
- Supone que los elementos pueden separarse (según algún criterio) en M categorías ordenadas.
- Es decir, para $i < j$, todo elemento de la cateoría i es menor que todo elemento de la categoría j.
- Idea:
  1. Construir un arreglo B de M listas y guardar los elementos de la categoría i en la i-ésima lista.
  2. Reconstruir el arreglo A, concatenando las listas en orden.

Bucket Sort

- $h(x)$ es una función que indica la categoría de x (de 0 a $M - 1$).
- Suponemos que se ejecuta en $O(1)$.

BUCKET-SORT(inout *A: Arreglo<T>, in M: int, in h: T → int*)

```
n ← length(A)
B ← arreglo de M listas vacías  // O(M)
for i ← [0..n − 1] do // O(n * costo cuerpo ciclo)
    insertar A[i] en la lista B[h(A[i])] // O(1) + costo de h
end for
concatenar listas B[0], B[1], . . ., B[M − 1]
```

- ¿Complejidad? $O(n + M)$, con n = |A|.

No hace falta escribir h, pero si explicarla en lenguaje natural.

Counting Sort

- **Counting Sort** asume que los elementos de un arreglo A de naturales son menores que un cierto valor k.
- Idea:
  1. Construir un arreglo B de tamaño k y guardar en la posición i, la cantidad de apariciones de i en A.
  2. Reconstruir el arreglo A, poniendo tantas copias de cada valor $j \in \{0..k\}$, como lo indique $B[j]$.
- **Observación:** Para un arreglo cualquiera de naturales, puede tomarse $k = \max_i \{A[i]\} + 1$.
- **Observación 2:** Se puede adaptar para el caso en que los valores están contenidos en un intervalo $[d, d + k)$.

Counting Sort (versión para naturales)

COUNTING-SORT-NAT(A: arreglo(Nat), max: Nat):

```
intervaloSize ← max + 1
B ← arreglo de tamaño intervaloSize
for i ← [0, length(B)−1] do // O(max)
    B[i] ← 0
    end for
    // cuento la cantidad de apariciones de cada elemento
    for i ← [0..length(A)−1] do // O(n)
    B[A[i]] ← B[A[i]] + 1
    end for
    // inserto en A la cantidad de apariciones de cada elemento
    indexA ← 0
    for i ← [0..length(B)−1] do // O(max+n)
    for j ← [1..B[i]] do
        A[indexA] ← i
        indexA ← indexA + 1
    end for
    end for
```

¿Complejidad? $O(n + k)$, con $n = |A|$

Counting Sort: Generalización

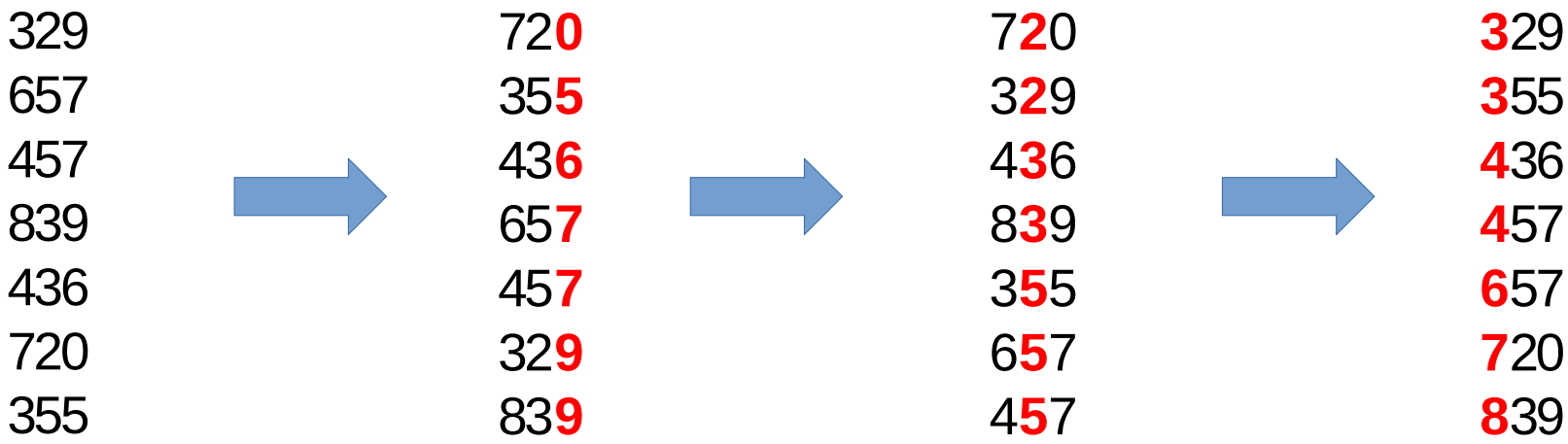
COUNTING-SORT(A: arreglo(int), min: int, max: int):

```
intervaloSize ← max − min + 1
B ← arreglo de tamaño intervaloSize
for i ← [0, length(B)−1] do // O(max−min)
    B[i] ← 0
    end for
    // cuento la cantidad de apariciones de cada elemento
    for i ← [0..length(A)−1] do // O(n)
    B[A[i] − min] ← B[A[i] − min] + 1
    end for
    // inserto en A la cantidad de apariciones de cada elemento
    indexA ← 0
    for i ← [0..length(B)−1] do // O(max−min + n)
    for j ← [1..B[i]] do
        A[indexA] ← i + min
        indexA ← indexA + 1
    end for
    end for
```

La complejidad es la misma

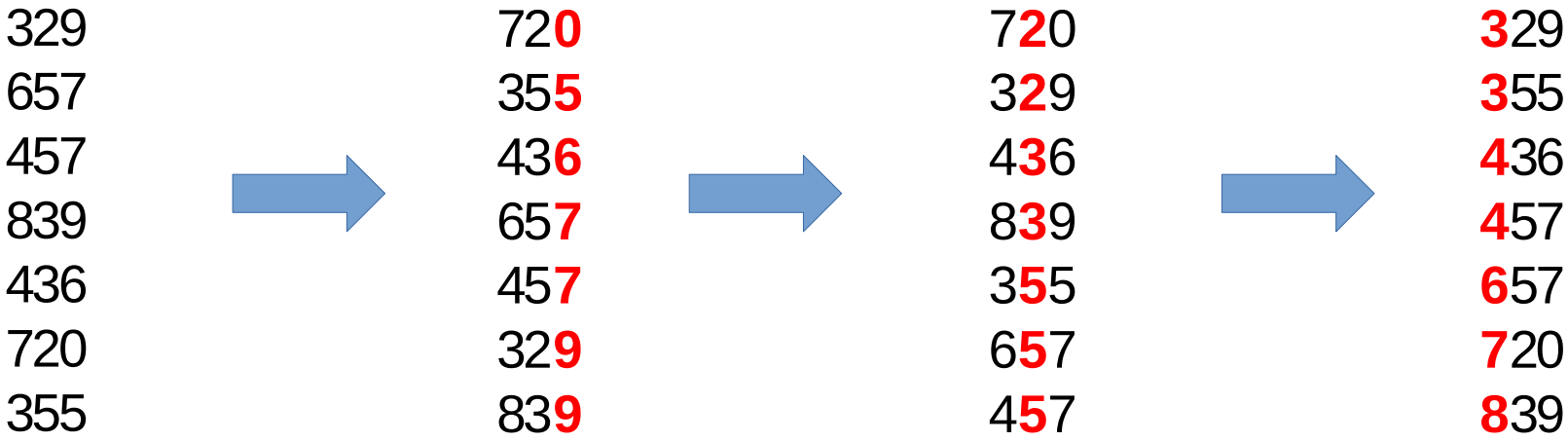
Radix Sort (versión para ordenar naturales)

- **Radix Sort**, en su versión más básica, ordena una lista de naturales.
- **Idea:**
  1. Ordenar los valores mirando sólo el dígito 1
  2. Ordenar los valores mirando sólo el dígito 2 (manteniendo el orden en caso de empate)
  3. Ordenar los valores mirando sólo el dígito 3 (manteniendo el orden en caso de empate)
  4. ...
- Ejemplo:



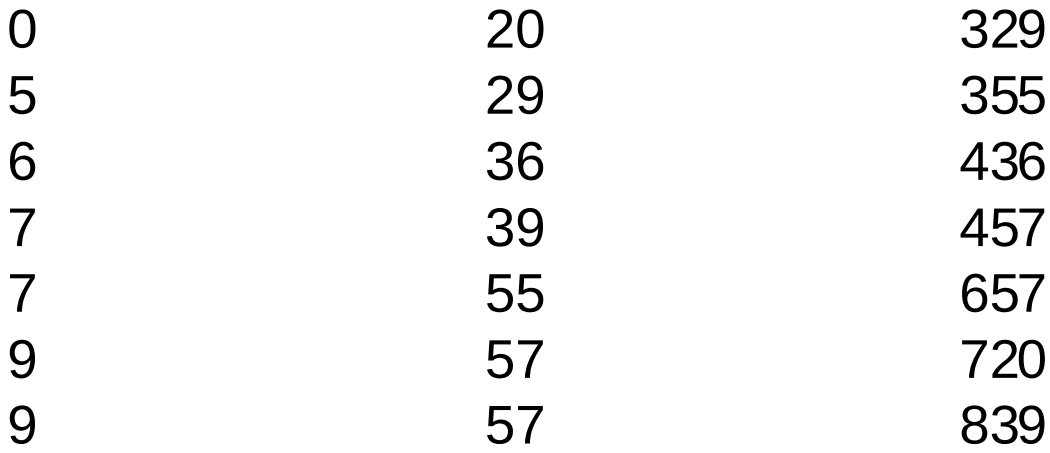
Radix Sort (versión para ordenar naturales)

Notar que a cada paso, la estabilidad hace que los números queden ordenados si los leemos a partir del dígito rojo, por lo que eventualmente quedan totalmente ordenados.



Invariante del ciclo de Radix:
En el paso i, quedan ordenados por sus I últimso dígitos ignorando los restantes.





Radix Sort (versión para ordenar naturales)

- Llamamos *d* a la cantidad máxima de dígitos de los elementos y suponemos que el dígito 1 es el menos significativo de cada valor.

RADIX-SORT(inout A: array(nat), in d: int)

---

for *i* ← [1..*d*] // del menos significativo al más significativo **do**
  Ordenar el arreglo *A* según el dígito *i*, en forma estable (usaremos bucket)
end for

- Recordemos que un algoritmo es **estable** si preserva el orden original entre las cosas equivalentes.
- ¿Complejidad? *d* × *O*(ordenar *A* por un dígito).
- Con Bucket Sort: O(Bucket * d) = O((n+m) * d) =
  O (n * d) //asumiendo un alfabeto acotado, ej: 0...9.

Radix Sort (generalizando)

- Radix Sort puede usarse también para ordenar cadenas de caracteres (la primera letra es la más significativa).
- El mismo esquema sirve para ordenar tuplas, donde el orden que se quiera dar depende principalmente de un “dígito” (i.e., un campo de la tupla) y en el caso de empate se tengan que revisar los otros.
- La idea en el fondo es la misma: usar un algoritmo estable e ir ordenado por “dígito” (del menos al más significativo).

RADIX-SORT(inout A: array<T>, in d: int)

---

for *i* ← [1..*d*] // del menos significativo al más significativo **do**
    Ordenar el arreglo *A* según el dígito *i*, en forma estable
end for

- La complejidad en este caso es $d \times O(\text{ordenar } A \text{ por un dígito})$.

Sobre los ejercicios de sorting

Algunas aclaraciones:

- Requieren práctica y paciencia.
- También requieren saber un poco de estructuras de datos.
- NO requieren que diseñen dichas estructuras (a menos que no sean las de siempre; como cuando hacen los ejercicios de elección de estructuras).
- Cada línea de código que escriban debería estar acompañada de la correspondiente complejidad temporal.

Sobre los ejercicios de sorting

Algunas estrategias:

- Utilizar algoritmos ya conocidos (Merge Sort, Quick Sort, Counting Sort, Bucket Sort, Radix Sort, etc.)
- Utilizar estructuras de datos ya conocidas (AVL, Heap, Trie, listas enlazadas, etc.)
- Analizar la complejidad pedida y deducir algo de eso.
- Si conocemos algo de la entrada, ver cómo se puede usar para mejorar la complejidad.

Ejercicios

Se desea ordenar los datos generados por un sensor industrial que monitorea la presencia de una sustancia en un proceso químico. Cada una de estas mediciones es un **número entero positivo**. Dada la naturaleza del proceso se sabe que, dada una secuencia de n mediciones, **a lo sumo ⌊√n⌋ valores están fuera del rango [20, 40]**.

Proponer un algoritmo O(n) que permita ordenar ascendentemente una secuencia de mediciones y justificar la complejidad del algoritmo propuesto.

Dado un **arreglo de pares <estudiante, nota>**, queremos **devolver** un **arreglo de pares <estudiante,promedio>**, ordenado en forma descendente por promedio y, en caso de empate, por número de libreta de forma ascendente.

Los estudiantes se representan con un **número de libreta** (puede considerar que es un número de **a lo sumo 10 dígitos**), y la **nota es un número entre 1 y 100**. **La cantidad de estudiantes se considera no acotada**.

Se pide dar un algoritmo que resuelva el problema con complejidad **O(n + m log m)**, donde n es la cantidad total de notas (es decir, el tamaño del arreglo de entrada), y m es la cantidad total de estudiantes.

Ejemplo:

Entrada: <12395, 72>, <45615, 81>, <12395, 94>, <45615, 100>, <78920, 81>,

<12395, 90>, <45615, 71>, <78920, 50>

Salida: <12395, 85.3333>, <45615, 84>, <78920, 65.5>

Resueltos

OrdenarMediciones(in array<int> A): array<int>
var vector<int> enRango //O(1)
var vector<int> fueraDeRango //O(1)
var array<int> res //O(1)
for i ←[0; |A|-1] do //O(n)
  if A[i]≥20 && A[i]≤40 then enRango.agregarAtras(A[i])//O(1)
  else fueraDeRango.agregarAtras(A[i])//O(1)
end for
MergeSort(fueraDeRango, comp)// comp es <
//O(n' . log(n')) = O(√(n) . log(√(n)) ϵ O(√(n)²) = O(n)
BucketSort(enRango, 21, h) // h= elem -20
//O(n)
res ← merge(fueraDeRango, en Rango)//O(n)

Orden total: O(n) +O(n) + O(n) + O(n) = **O(n)**

```
OrdenarXPromedio(in array<(<int>,<int>)> A): array<(<int>,<float>)>)
var vector<(<int>,<float>)> res ← []
RadixSort(A, 10)//ordeno por LU
if |A| != 0
  var int estudianteActual ← A[0][0]  //O(1)
  var int notasAcumuladas ← A[0][1]  //O(1)
  var int cantidadNotas ← 1 //O(1)
  var float promedio //O(1)
  var int actual ←1 //O(1)
  while actual < |A| //n veces → O(n)
    if A[actual][0] == estudianteActual //O(1)
      notasAcumuladas ← notasAcumuladas + A[actual][1] //O(1)
      CantidadNotas++ //O(1)
    else
      promedio ← notasAcumuladas/cantidadNotas //O(1)
      res.agregarAtras(<estudianteActual, promedio>) //O(1)
      estudianteActual ← A[actual][0] //O(1)
      notasAcumuladas ← A[actual][1] //O(1)
      CantidadNotas ← 1 //O(1) endif
    actual++  endwhile //(omito enter porque sino no entra el algoritmo, mismo en línea anterior)
 promedio ← notasAcumuladas/cantidadNotas //O(1)
 res.agregarAtras(<estudianteActual, promedio>) //O(1)
 mergeSort(res, comp)   //O(m log(m))  comp = mayor promedio, menor LU en caso de empate
endif
```

Orden total: O(n) + O(m log(m))

Ejercicio de Parcial (yapa)

Se está por realizar un gran censo de perros en la Ciudad de Buenos Aires. La información recaudada llegará en forma de un arreglo de **tuplas <Nombre, Raza, Edad, Peso>**. Las **razas son strings de largo acotado L**, pero **no sabemos cuántas razas pueden existir**, aunque sí sabemos que entre ellas está la mejor raza de todas: raza Perro. Las **edades** serán **enteros positivos entre 1 y 20** y los **pesos** serán de tipo **float**.

Proponga un algoritmo que organice la información en dos arreglos:
- Uno con los perros **raza Perro** ordenados **por edad y**, en caso de empate, **por peso**, ambos en forma ascendente
- Otro arreglo con la información de las **razas** y la **cantidad de perros** de cada una en forma de tuplas <Raza, int>, ordenadas en **forma decreciente** por la cantidad y, **en caso de empate, alfabéticamente por la raza**

La complejidad temporal del algoritmo debe ser O(n + p*log(p)), donde n es el total de perros y p el total de perros raza Perro.

Ejemplo Si recibimos:
[<Nulus, Perro, 4, 15.3>, <Joe, Cocker, 14, 8.0>, <Lady, Caniche, 10, 6.1>, <Dobby, Perro, 7, 18.3>, <Res, Dogo, 3, 50.0>, <Haru, Perro, 7, 30.0>, <Clementina, Perro, 14, 10.0>, <Choqui, Caniche, 16, 5.0>]

Deberíamos devolver:
P: [<Nulus, 4, 15.3>, <Dobby, 7, 18.3>, <Haru, 7, 30.0>, <Clementina, 14, 10.0>]
R: [<Caniche, 2>, <Cocker, 1>, <Dogo, 1>]